correctly and resist the temptation to circumvent it unless you're certain it's safe to do so.

Section 29.3 deals with a common scenario: you have an application that wants to use JavaScript requests to retrieve data from a second app. By default, web browsers block requests to other apps, so you need to enable cross-origin resource sharing (CORS) in your API to achieve this. We'll look at how CORS works, how to create a CORS policy for your app, and how to apply it to specific endpoints.

The final section of this chapter, section 29.4, covers a collection of common threats to your application. Each one represents a potentially critical flaw that an attacker could use to compromise your application. The solutions to each threat are generally relatively simple; the important thing is to recognize where the flaws could exist in your own apps so you can ensure that you don't leave yourself vulnerable.

As I mentioned in chapter 28, you should always start by adding HTTPS to your app to encrypt the traffic between your users' browsers and your app. Without HTTPS, attackers could subvert many of the safeguards you add to your app, so it's an important first step to take.

Unfortunately, most other security practices require rather more vigilance to ensure that you don't accidentally introduce vulnerabilities into your app as it grows and develops. Many attacks are conceptually simple and have been known about for years, yet they're still commonly found in new applications. In the next section we'll look at one such attack and see how to defend against it when building apps using Razor Pages.